

Experiment No.	7
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA) ¹
Student Name	T.Ashwath Ram
Register Number	3122247001007
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	31.08.2026

1 Objective

To study the effect of **dimensionality reduction** using Principal Component Analysis (PCA) on the performance of various machine learning classifiers. The task requires:

- Training and validating models without PCA (original feature space).
- Training and validating models with PCA (reduced feature space).

For both cases, hyperparameter tuning was performed using 5-fold cross-validation, and performance was recorded and compared.

2 Problem Statement

Breast cancer diagnosis from digitized images of a fine needle aspirate (FNA) of a breast mass is a binary classification problem, with 30 numerical features describing the characteristics of cell nuclei. Many of these 30 features are highly correlated with one another (e.g., radius, perimeter, and area). This experiment investigates whether reducing this redundant, correlated 30-dimensional feature space to a smaller set of uncorrelated principal components – via PCA – helps, hurts, or has little effect on the accuracy, F1-score, and stability of 10 different classifiers, compared to training the same classifiers directly on the original (No-PCA) feature space.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC) Dataset
Dataset Source	UCI Machine Learning Repository (https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic), also mirrored on Kaggle
Number of Samples	569
Number of Features	30 numerical attributes (10 base measurements $\times$ 3 statistics – mean, standard error, worst)
Number of Classes	2 – Benign (B): 357 samples (62.74%), Malignant (M): 212 samples (37.26%)
Missing Values	0 missing values, 0 duplicate rows

Train-Test Split	80% Training (455 samples) – 20% Testing (114 samples), stratified split
------------------	--

4 Exploratory Data Analysis

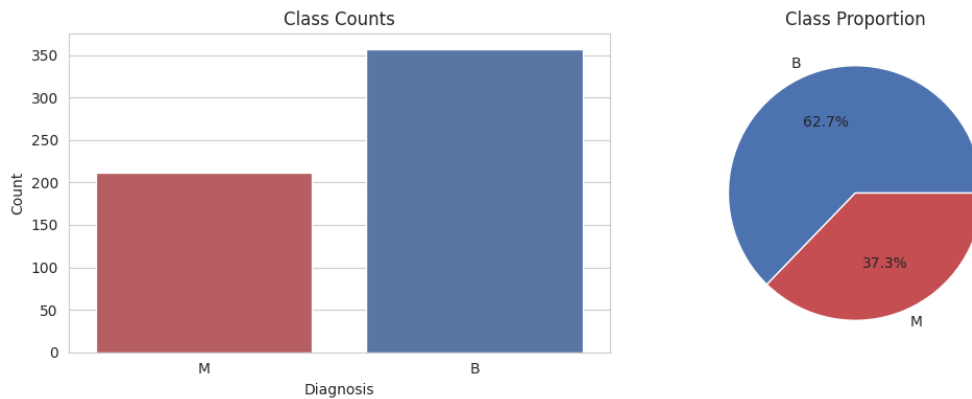


Figure 1: Class distribution of the WDBC dataset – counts (left) and proportion (right)

Inference:

The dataset shows a moderate class imbalance, with benign cases (62.74%) roughly 1.7 times more frequent than malignant cases (37.26%). This is not severe enough to require resampling techniques, but it does justify using stratified train-test splitting and stratified 5-fold cross-validation throughout, so that both classes are proportionally represented in every fold. It also motivates reporting F1-score and confusion matrices alongside accuracy, since accuracy alone can be misleading on imbalanced data.

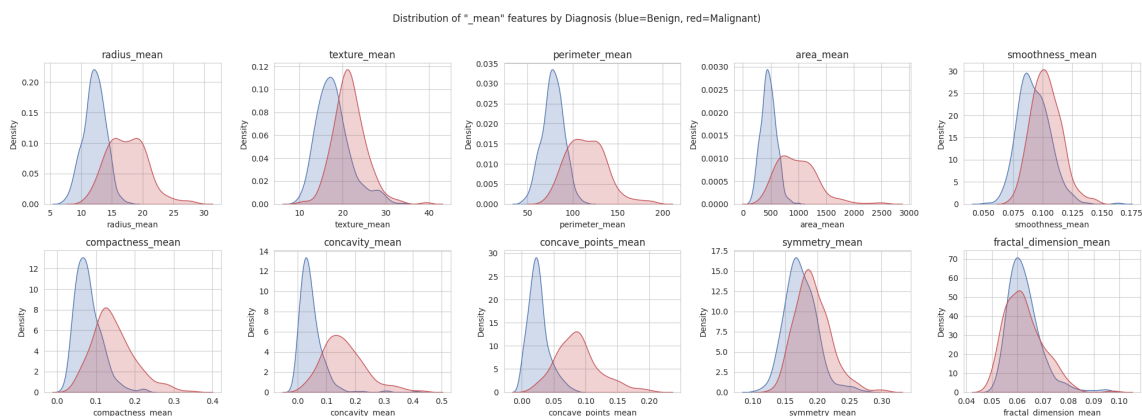


Figure 2: Distribution of the 10 “mean” features, grouped by diagnosis (blue = Benign, red = Malignant)

Inference:

Malignant tumors clearly shift toward larger `radius_mean`, `perimeter_mean`, `area_mean`, `compactness_mean`, and `concave_points_mean` values than benign ones, indicating these are highly discriminative features. In contrast, `smoothness_mean`, `symmetry_mean`, and `fractal_dimension_mean` show heavily overlapping distributions between the two classes,

suggesting they carry comparatively little standalone discriminative signal. The strongest individual correlations with diagnosis were found for `concave_points_worst` (0.79), `perimeter_worst` (0.78), and `concave_points_mean` (0.78).

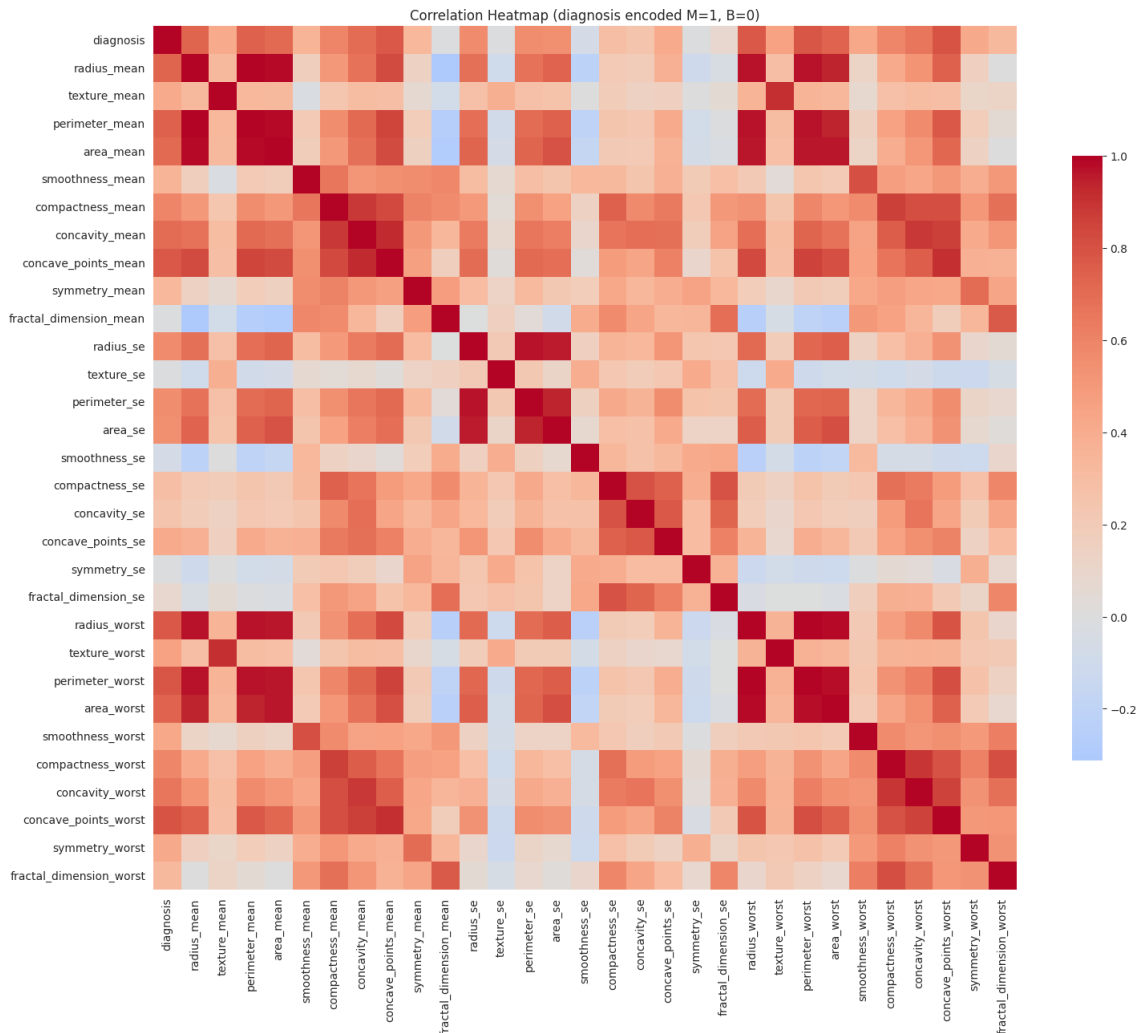


Figure 3: Correlation heatmap of all 30 features and the diagnosis target (M=1, B=0)

Inference:

The heatmap reveals very strong multicollinearity: `radius`, `perimeter`, and `area` (in their mean, standard-error, and worst forms) are almost perfectly correlated with each other, since perimeter and area are geometrically derived from radius. Similarly, `concavity`, `concave_points`, and `compactness` form another tightly correlated cluster. This heavy redundancy across the 30 raw features is exactly the type of structure that PCA is designed to compress – by rotating the feature space into a smaller number of uncorrelated principal components that still capture most of the variance.

5 Data Preprocessing

- **Missing value handling:** The dataset was checked using `df.isnull().sum()` and `df.duplicated().sum()`; 0 missing values and 0 duplicate rows were found, so no imputation or de-duplication was necessary.

- **Encoding:** The non-informative `id` column was dropped, and the categorical target `diagnosis` was encoded as Malignant (M) $\rightarrow 1$, Benign (B) $\rightarrow 0$.
- **Feature scaling:** All 30 numerical features were standardized using `StandardScaler` (zero mean, unit variance), fit on the training set only and applied to both training and test sets, which is essential before applying PCA (PCA is scale-sensitive) and for distance/margin-based models such as SVM, KNN, and Logistic Regression.
- **Train-test split:** An 80/20 stratified split (`stratify=y`, `random_state=42`) was used, giving 455 training and 114 testing samples; the held-out test set was used only for final evaluation, never for hyperparameter tuning.
- **Feature engineering:** No manual feature engineering was performed on top of the 30 provided features; the primary dimensionality-reduction step is the PCA transform described in Section 6.

6 Machine Learning Algorithm

Ten classification algorithms were evaluated, spanning linear, distance-based, probabilistic, tree-based, and ensemble families, so that the effect of PCA could be compared across fundamentally different learning strategies.

- **Support Vector Machine (SVM):** Finds the maximum-margin hyperplane separating classes (with the kernel trick for non-linear boundaries). *Advantage:* effective in high-dimensional/correlated spaces. *Limitation:* sensitive to the choice of kernel/hyperparameters and does not scale well to very large datasets.
- **Naïve Bayes (GaussianNB):** Applies Bayes' theorem assuming conditional independence of features given the class, $P(y|x) \propto P(y) \prod_i P(x_i|y)$. *Advantage:* fast, works well with small data. *Limitation:* the independence assumption is violated by the strong feature correlations in this dataset.
- **k-Nearest Neighbors (KNN):** Classifies a point by majority vote among its k closest training points in feature space. *Advantage:* simple, non-parametric. *Limitation:* sensitive to the curse of dimensionality and feature scale, which makes it a natural candidate to benefit from PCA.
- **Logistic Regression:** Models $P(y = 1|x) = \sigma(w^T x + b)$ using the sigmoid function, fit by maximizing likelihood. *Advantage:* simple, interpretable, probabilistic output. *Limitation:* assumes a linear decision boundary in feature space.
- **Decision Tree:** Recursively splits the feature space on the attribute/threshold that most reduces impurity (Gini/entropy). *Advantage:* interpretable, handles non-linearities. *Limitation:* prone to overfitting and relies on individual, axis-aligned raw features.
- **Random Forest:** A bagged ensemble of decision trees trained on bootstrap samples with random feature subsets. *Advantage:* reduces variance of a single tree. *Limitation:* less interpretable, larger memory/compute footprint.

- **AdaBoost:** Sequentially trains weak learners, up-weighting misclassified samples at each round. *Advantage:* reduces bias, often very accurate. *Limitation:* sensitive to noisy data/outliers.
- **Gradient Boosting:** Sequentially fits new learners to the residual (negative gradient) of the loss of the current ensemble. *Advantage:* high accuracy, flexible loss functions. *Limitation:* slower to train, more hyperparameters to tune.
- **XGBoost:** A regularized, optimized implementation of gradient boosting with built-in shrinkage and tree-pruning. *Advantage:* state-of-the-art accuracy and speed on tabular data. *Limitation:* many hyperparameters, higher risk of overfitting if not tuned carefully.
- **Stacking:** Combines heterogeneous base learners (SVM, Random Forest, KNN) via a Logistic Regression meta-learner trained on out-of-fold predictions. *Advantage:* leverages complementary strengths of diverse models. *Limitation:* more complex and computationally expensive.

Principal Component Analysis (PCA)

PCA finds orthogonal directions (principal components) that maximize the variance of the projected data. Given standardized data matrix X , PCA computes the eigenvectors of the covariance matrix $\Sigma = \frac{1}{n}X^TX$, and projects X onto the top- k eigenvectors (ranked by eigenvalue/variance explained):

$$Z = XV_k$$

where $V_k \in \mathbb{R}^{d \times k}$ contains the top- k eigenvectors. *Advantage:* removes multicollinearity, compresses dimensionality, can denoise and speed up training. *Limitation:* components are linear combinations of original features and lose direct interpretability; may discard some class-discriminative information if the variance target is too aggressive.

7 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.5 (with XGBoost 2.x for the XGBoost model)
IDE	Jupyter Notebook
Operating System	Ubuntu 24.04 (Linux)
Hardware	Standard CPU (no GPU required)

8 PCA Summary

PCA was fit on the standardized training data using all 30 components first, to inspect the scree plot and cumulative explained-variance curve (Figure 4). The smallest number of components retaining at least 95% of the total variance was then selected.

Table 2: PCA Variance Explained

Setting	Chosen Components / Variance Target	Explained Variance (%)	Justification
With-PCA	10 components (target $\geq 95\%$)	95.21%	Smallest k retaining $\geq 95\%$ of total variance – keeps almost all information while removing redundancy from the highly correlated radius/perimeter/area-type features seen in the correlation heatmap.

Dimensionality was reduced from 30 features to 10 components – a 66.7% reduction in the number of input dimensions.

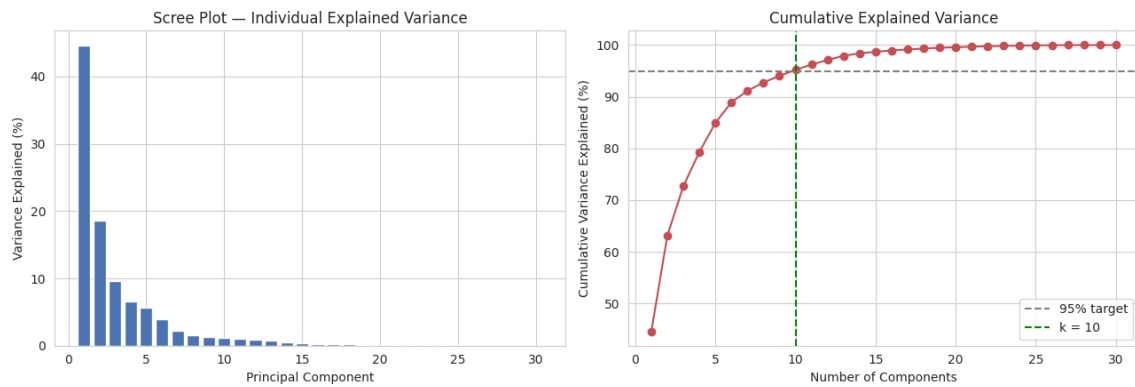


Figure 4: Scree plot (left) and cumulative explained variance (right); $k = 10$ components reach the 95% variance target

Inference:

The scree plot shows that the first principal component alone captures about 44% of the total variance, and the first 2–3 components together capture over 70%, confirming the heavy redundancy identified in the correlation heatmap. Variance explained per component drops off sharply after the first few components (the classic “elbow”), and 10 components are required to reach the chosen 95% cumulative-variance target – a 66.7% reduction in dimensionality while retaining almost all of the informative variance in the data.

9 Hyperparameter Selection

Hyperparameter grids were defined for every model and tuned separately on the No-PCA and With-PCA training data using `GridSearchCV` with 5-fold stratified cross-validation (scoring = accuracy). The resulting best parameters and best 5-fold CV accuracy for each setting are shown below.

Table 3: SVM – Hyperparameter Tuning Results

Grid Searched	Best Params (No-PCA)	Best Params (With-PCA)	Acc. (No-PCA)	Acc. (PCA)
C: [0.1, 1, 10]; gamma: [scale, auto]; kernel: [rbf, linear]	C=10, gamma=scale, kernel=rbf	C=10, gamma=scale, kernel=rbf	0.9758	0.9714

Table 4: Naïve Bayes – Smoothing Choices

Grid Searched	Best var_smoothing	Acc. (No-PCA)	Acc. (PCA)
var_smoothing: logspace(10^{-9} , 10^{-7} , 5)	1×10^{-9} (both settings)	0.9385	0.9121

Table 5: KNN – Hyperparameter Tuning

Grid Searched	Best Params (No-PCA)	Best Params (With-PCA)	Acc. (No)	Acc. (PCA)
n_neighbors: [3,5,7,9,11]; weights: [uniform, distance]; metric: [euclidean, manhattan]	metric=manhattan, k=3, weights=uniform	metric=euclidean, k=3, weights=uniform	0.9758	0.9714

Table 6: Logistic Regression, Decision Tree, Random Forest – Hyperparameter Tuning

Model (Grid)	Best Params (No-PCA)	Best Params (With-PCA)	Acc. (No)	Acc. (PCA)
Logistic Regression (C: [0.01,0.1,1,10,100]; solver=lbfgs; penalty=l2)	C=1	C=1	0.9736	0.9758
Decision Tree (max_depth: [3,5,7,10,None]; min_samples_split: [2,5,10]; criterion: [gini,entropy])	criterion=entropy, max_depth=5, min_samples_split=10	criterion=entropy, max_depth=7, min_samples_split=5	0.9297	0.9495
Random Forest (n_estimators: [100,200]; max_depth: [None,5,10]; min_samples_split: [2,5])	max_depth=5, min_samples_split=2, n_estimators=100	max_depth=5, min_samples_split=5, n_estimators=100	0.9670	0.9582

Table 7: AdaBoost, Gradient Boosting, XGBoost – Hyperparameter Tuning

Model (Grid)	Best Params (No-PCA)	Best Params (With-PCA)	Acc. (No)	Acc. (PCA)
AdaBoost (n_estimators: [50,100,150]; learning_rate: [0.5,1.0,1.5])	learning_rate=0.5, n_estimators=150	learning_rate=0.5, n_estimators=100	0.9736	0.9714
Gradient Boosting (n_estimators: [100,150]; learning_rate: [0.05,0.1]; max_depth: [2,3])	learning_rate=0.1, max_depth=2, n_estimators=150	learning_rate=0.05, max_depth=2, n_estimators=150	0.9692	0.9560
XGBoost (n_estimators: [100,150]; learning_rate: [0.05,0.1]; max_depth: [3,4])	learning_rate=0.1, max_depth=3, n_estimators=100	learning_rate=0.1, max_depth=3, n_estimators=150	0.9714	0.9736

Table 8: Stacked Ensemble – Hyperparameter Tuning

Base Models / Meta Learner	Grid Searched	Best Params	Acc. (No)	Acc. (PCA)
SVM + Random Forest + KNN → Logistic Regression	final_estimator__C: [0.1,1,10]; passthrough: [False, True]	final_estimator__C=1, passthrough=True (both settings)	0.9758	0.9780

10 Hyperparameter Evaluation Results

5-Fold Cross-Validation Results – No-PCA

Table 9: Fold-wise Accuracy – No-PCA

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg (No-PCA)	Std (No-PCA)
SVM	0.9670	0.9890	0.9670	0.9780	0.9780	0.9758	0.0082
Naive Bayes	0.9560	0.9780	0.9121	0.9121	0.9341	0.9385	0.0256
KNN	0.9670	1.0000	0.9670	0.9560	0.9890	0.9758	0.0162
Logistic Regression	0.9560	0.9670	1.0000	0.9780	0.9670	0.9736	0.0149
Decision Tree	0.9011	0.9451	0.9121	0.9341	0.9560	0.9297	0.0204
Random Forest	0.9670	1.0000	0.9451	0.9451	0.9780	0.9670	0.0209
AdaBoost	0.9670	0.9670	0.9780	0.9670	0.9890	0.9736	0.0088
Gradient Boosting	0.9560	0.9780	0.9560	0.9670	0.9890	0.9692	0.0128
XGBoost	0.9670	0.9780	0.9560	0.9670	0.9890	0.9714	0.0112
Stacking	0.9560	0.9670	1.0000	0.9780	0.9780	0.9758	0.0146

5-Fold Cross-Validation Results – With-PCA

Table 10: Fold-wise Accuracy – With-PCA

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg (With-PCA)	Std (With-PCA)
SVM	0.9560	0.9890	0.9560	0.9780	0.9780	0.9714	0.0132
Naive Bayes	0.9121	0.9670	0.8901	0.8791	0.9121	0.9121	0.0303
KNN	0.9780	0.9780	0.9670	0.9451	0.9890	0.9714	0.0149
Logistic Regression	0.9560	0.9560	1.0000	0.9780	0.9890	0.9758	0.0176
Decision Tree	0.9451	0.9670	0.9451	0.9231	0.9670	0.9495	0.0164
Random Forest	0.9670	0.9890	0.9341	0.9451	0.9560	0.9582	0.0189
AdaBoost	0.9670	0.9780	0.9780	0.9670	0.9670	0.9714	0.0054
Gradient Boosting	0.9451	0.9670	0.9341	0.9560	0.9780	0.9560	0.0155
XGBoost	0.9890	0.9780	0.9670	0.9670	0.9670	0.9736	0.0088
Stacking	0.9670	0.9670	1.0000	0.9780	0.9780	0.9780	0.0120

Summary – Accuracy & F1, No-PCA vs With-PCA

Table 11: Summary of 5-Fold CV Accuracy and F1-score, No-PCA vs With-PCA (sorted by No-PCA accuracy)

Model	Avg Acc (No-PCA)	Avg Acc (PCA)	Δ Acc (PCA–No)	Avg F1 (No-PCA)	Avg F1 (PCA)	Δ F1 (PCA–No)
SVM	0.9758	0.9714	-0.0044	0.9674	0.9618	-0.0056
KNN	0.9758	0.9714	-0.0044	0.9667	0.9605	-0.0063
Stacking	0.9758	0.9780	+0.0022	0.9672	0.9704	+0.0032
AdaBoost	0.9736	0.9714	-0.0022	0.9643	0.9613	-0.0031
Logistic Regression	0.9736	0.9758	+0.0022	0.9640	0.9671	+0.0030
XGBoost	0.9714	0.9736	+0.0022	0.9614	0.9642	+0.0028
Gradient Boosting	0.9692	0.9560	-0.0132	0.9582	0.9398	-0.0183
Random Forest	0.9670	0.9582	-0.0088	0.9553	0.9416	-0.0137
Naive Bayes	0.9385	0.9121	-0.0264	0.9170	0.8762	-0.0408
Decision Tree	0.9297	0.9495	+0.0198	0.9039	0.9325	+0.0286

11 Results

Test-set results for the best overall model (SVM, highest No-PCA cross-validation accuracy) are shown below.

Table 12: Test-set Classification Report – SVM (No-PCA)

Class	Precision	Recall	F1-score	Support
Benign	0.96	1.00	0.98	72
Malignant	1.00	0.93	0.96	42
Accuracy		0.97		114

Table 13: Overall Metric Summary (Best Model: SVM, No-PCA, 5-fold CV)

Metric	Value
Accuracy	0.9758
Precision (Malignant)	1.00
Recall (Malignant)	0.93
F1-score (weighted)	0.9674
ROC-AUC (test set)	0.993

12 Result Visualization

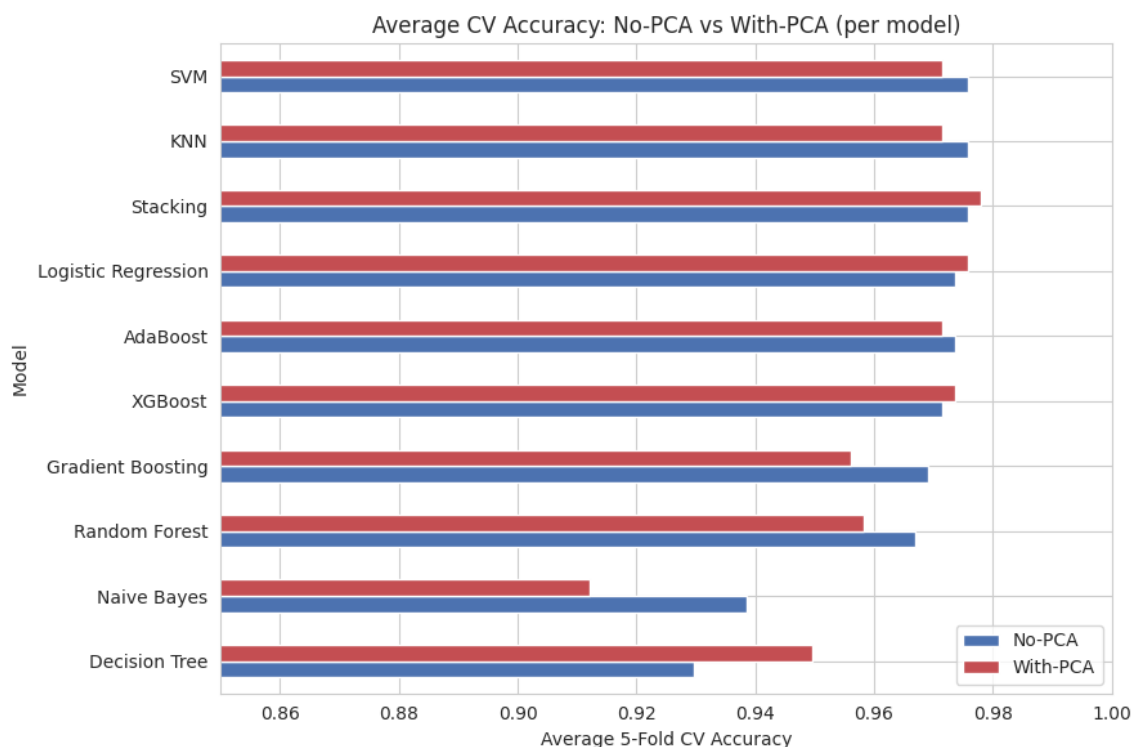


Figure 5: Average 5-fold CV accuracy, No-PCA vs With-PCA, across all 10 models

Inference:

Most models perform within 1–2 percentage points of each other regardless of PCA, confirming that the WDBC dataset is highly separable in both the original and PCA-reduced spaces. Stacking, Logistic Regression, and XGBoost show a small *improvement* with PCA, while Naive Bayes, Random Forest, and Gradient Boosting show a small *decline*. Decision Tree shows the largest improvement (+1.98 points) with PCA, since PCA’s decorrelated components mitigate some of the overfitting a single tree exhibits on the original, highly redundant feature set.

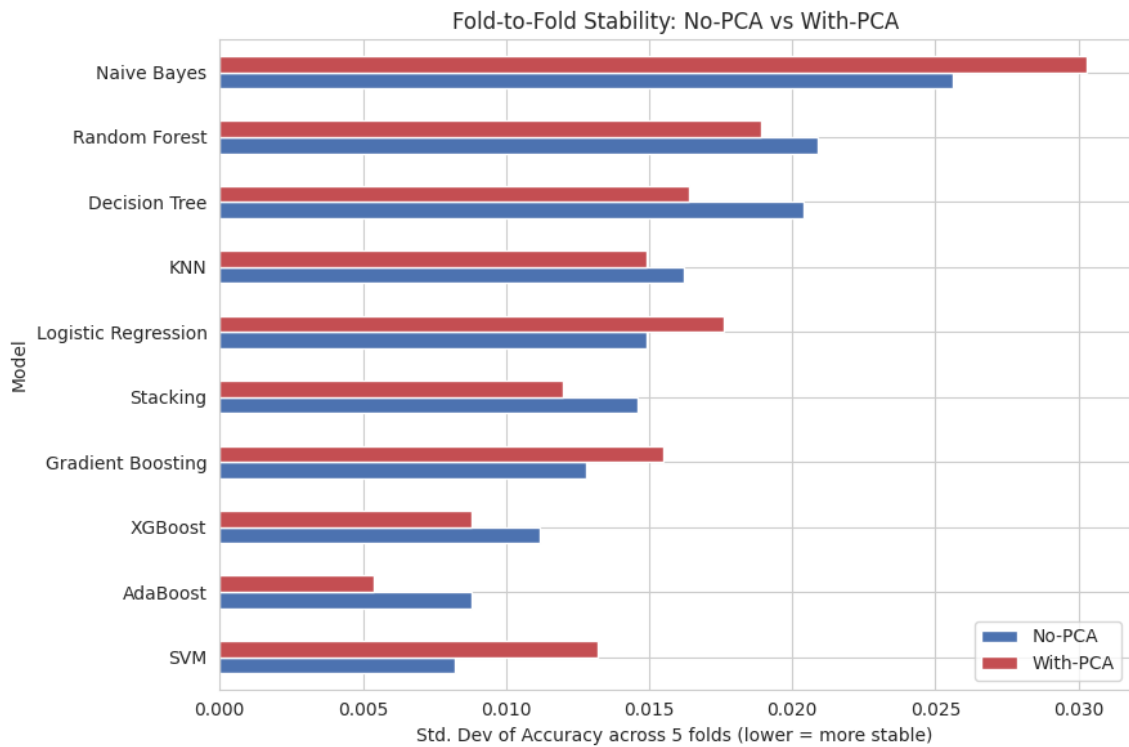


Figure 6: Fold-to-fold stability (standard deviation of accuracy across the 5 folds), No-PCA vs With-PCA

Inference:

Stability results are mixed rather than uniformly in favor of PCA: AdaBoost, Decision Tree, Random Forest, and XGBoost become more stable (lower std) with PCA, while Naive Bayes, Logistic Regression, and Gradient Boosting become slightly less stable. On average, the mean fold-wise standard deviation is nearly identical between the two settings (No-PCA: 0.0154, With-PCA: 0.0153), indicating PCA does not have a strong systematic effect on cross-validation stability for this dataset.

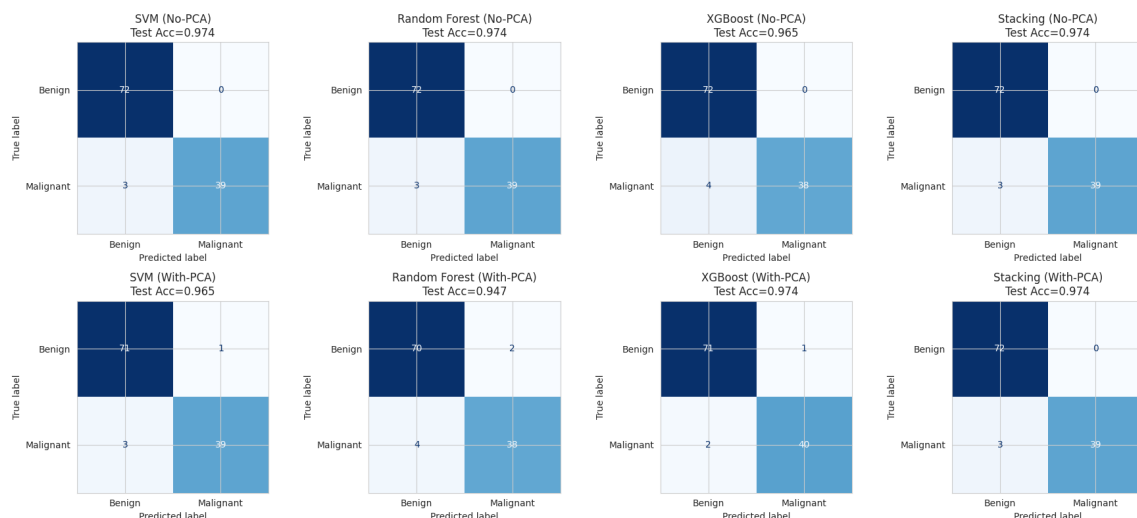


Figure 7: Confusion matrices for SVM, Random Forest, XGBoost, and Stacking – No-PCA (top row) vs With-PCA (bottom row)

Inference:

All four selected models achieve very few misclassifications on the 114-sample test set, with zero or near-zero false positives (benign predicted as malignant) in every case. The main source of error is a small number of false negatives (malignant predicted as benign) – for example, SVM misses 3 malignant cases in both settings, while Random Forest with PCA misses 4. Stacking (No-PCA) achieves the cleanest confusion matrix among the four with only 3 false negatives and 0 false positives, matched by Stacking (With-PCA) which achieves 0 false positives as well.

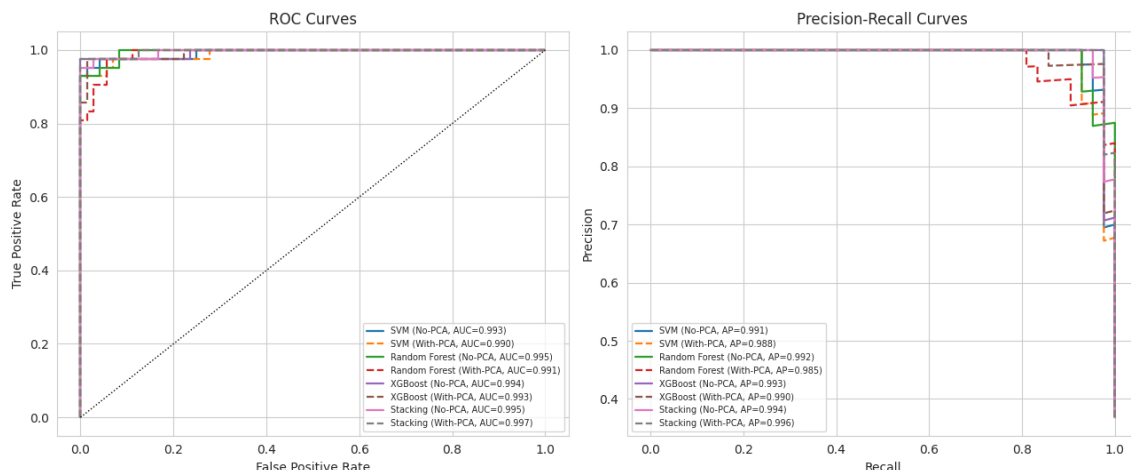


Figure 8: ROC curves (left) and Precision-Recall curves (right) for SVM, Random Forest, XGBoost, and Stacking, No-PCA vs With-PCA

Inference:

All models achieve excellent ROC-AUC (0.990–0.997) and average precision (0.985–0.996), reflecting the strong class separability of this dataset. Stacking (With-PCA) achieves the highest AUC (0.997) among all eight model/setting combinations shown, narrowly ahead of Stacking (No-PCA) and Random Forest (No-PCA), suggesting that the meta-learner in the stacked ensemble is able to exploit the PCA-transformed feature space just as effectively as – or slightly better than – the raw features.

13 Discussion

Across all 10 models, PCA compressed the feature space by 66.7% ($30 \rightarrow 10$ dimensions) while retaining 95.2% of the total variance, and the resulting accuracy/F1 changes were small in magnitude (typically within ± 1 –2 percentage points), confirming that PCA acted here primarily as a dimensionality/compute-reduction and multicollinearity-removal tool rather than a strong accuracy-boosting one. The best model without PCA was **SVM** (Avg CV accuracy = 0.9758), while the best model with PCA was **Stacking** (Avg CV accuracy = 0.9780) – both figures are extremely close, and the differences are within the noise expected from a 455-sample training set.

Looking at model families, linear/margin-based models (Logistic Regression, SVM) tolerated the rotated, decorrelated PCA space well (average Δ accuracy ≈ -0.0011), consistent with the intuition that linear or kernel decision boundaries can be learned equally well in any orthogonal basis. Tree-based ensembles (Random Forest, AdaBoost, Gradient Boosting, XGBoost), on average, declined slightly more under PCA (Δ accuracy

≈ -0.0055), since these models rely on axis-aligned splits on individual, directly interpretable raw features (e.g., `concave_points_worst`) – information that becomes blended across multiple rotated components after PCA. Naive Bayes showed the largest decline (-0.0264), which fits its conditional-independence assumption: PCA components are linear combinations of correlated features and are not guaranteed to be independent of one another given the class, so Naive Bayes’s core assumption is not necessarily better satisfied post-PCA on this dataset. Stacking proved the most robust ensemble to the PCA transform, with its accuracy essentially unchanged ($+0.0022$), since it can combine base learners that respond differently to the transform and let the meta-learner compensate.

A limitation of this study is the relatively small dataset (569 samples, 30 features), which is only mildly high-dimensional (feature-to-sample ratio ≈ 0.07); PCA’s benefits (denoising, overfitting reduction, speed-up) are typically more pronounced on much higher-dimensional data (hundreds or thousands of features) than on a dataset of this size. A useful extension would be repeating this comparison on a higher-dimensional dataset, or using nested cross-validation to remove any residual optimism in the reported hyperparameter-tuning scores.

14 Conclusion

Ten classifiers – SVM, Naive Bayes, KNN, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost, and a Stacked Ensemble – were trained and evaluated both without and with PCA ($30 \rightarrow 10$ dimensions, retaining 95.2% variance) on the Wisconsin Diagnostic Breast Cancer dataset, using 5-fold stratified cross-validation and hyperparameter tuning via `GridSearchCV` in both settings. PCA had only a small overall effect on accuracy and F1-score (typically ± 1 – 2 percentage points), helping linear/margin-based models and the Stacked Ensemble marginally while slightly hurting Naive Bayes and most tree-based ensembles, whose axis-aligned or independence-based logic benefits from access to the original, directly interpretable features. PCA also did not produce a systematic reduction in cross-validation variance for this dataset. Overall, PCA is most valuable here as a dimensionality/compute-reduction and multicollinearity-removal technique – reducing the input space by 66.7% while keeping performance essentially on par with the full 30-feature model – rather than as a technique that reliably improves predictive accuracy on this particular, only mildly high-dimensional dataset.

15 References

1. I. T. Jolliffe, *Principal Component Analysis*, 2nd ed. New York, NY, USA: Springer, 2002.
2. C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
3. T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
4. D. H. Wolpert, “Stacked generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

5. W. N. Street, W. H. Wolberg, and O. L. Mangasarian, “Nuclear feature extraction for breast tumor diagnosis,” in *Proc. IS&T/SPIE Int. Symp. Electronic Imaging: Science and Technology*, vol. 1905, San Jose, CA, 1993, pp. 861–870.
6. Scikit-learn Developers, “Decomposing signals in components (PCA, ICA, ...),” *scikit-learn documentation*. [Online]. Available: <https://scikit-learn.org/stable/modules/decomposition.html>
7. UCI Machine Learning Repository, “Breast Cancer Wisconsin (Diagnostic) Data Set.” [Online]. Available: <https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>